

# 1. 题目深度解构

## 题意重述

给定一棵  $n$  个点的树。初始时只有一个点  $s$  被标记。Alice 和 Bob 轮流操作。

每次操作需要标记 1 到  $k$  个尚未标记的点，并且这些点必须按顺序形成一条路径：

第一个点必须与某个已标记点相邻；之后每个新标记点都必须与上一个新标记点相邻。

谁标记最后一个未标记点，谁获胜。

要求对每个初始点  $s = 1, 2, \dots, n$ ，判断先手 Alice 是否必胜。

## 约束分析

$$n \leq 2000, \quad \sum n \leq 2000$$

这个范围提示我们可以接受：

$$O(n^2)$$

甚至带较小常数的  $O(n^2 \log n)$ 。

但不能暴力枚举游戏状态。因为标记点集合理论上有：

$$2^n$$

种状态。

## 题目类型判定

这是典型的：

树上公平组合游戏 + Sprague-Grundy 博弈 + 有向边 DP

核心不是普通树形 DP，而是要把“当前已标记区域”切分成若干独立子游戏，然后用 SG 函数异或合并。

# 2. 思维路径推导

## 从暴力状态开始想

如果直接把一个状态表示成“哪些点已经被标记”，那么状态数是  $2^n$ ，完全不可行。

但题目有一个非常强的结构限制：

每次新标记的点是一条从已标记区域向外延伸的路径。

因此，已标记点集合始终是连通的。

证明很简单：

初始只有一个点，连通。每次操作添加一条路径，并且这条路径的第一个点与已标记区域相邻，所以添加后仍然连通。

## 关键观察：剩余部分会分裂成独立子游戏

因为整张图是一棵树，当已标记区域是一个连通子树时，删掉它之后，剩下的每个连通块都只会通过一个边界点接到已标记区域。

一次操作只能从某一个边界连通块开始，沿着一条路径往里面标记若干点。

也就是说：

每一步操作只会影响一个剩余连通块，其它连通块完全不变。

这正是 SG 博弈中“子游戏异或”的结构。

所以我们不应该记录整个已标记集合，而应该研究一个挂在已标记区域外面的“树枝”作为一个独立游戏。

## 定义有向边状态

考虑一条有向边：

$$p \rightarrow u$$

其中  $p$  已经被标记， $u$  还没有被标记。我们只看从  $u$  出发、不经过  $p$  的那一侧子树。

定义：

$$F(u, p)$$

表示这个子游戏的 SG 值。

也就是说，当前  $p$  是已标记区域的一部分，玩家下一步如果选择这个子游戏，就必须从  $u$  开始标记。

## 如果只标记一个点

假设当前玩家在子游戏  $F(u, p)$  中只标记  $u$  一个点。

那么标记完  $u$  后， $u$  的其它邻居  $v \neq p$  各自变成新的独立子游戏：

$$F(v, u)$$

所以这个操作后的 SG 值为：

$$G(u, p) = \bigoplus_{v \in \text{adj}[u], v \neq p} F(v, u)$$

这里  $G(u, p)$  可以理解为：

标记掉  $u$  之后，剩余分支的 SG 异或和。

## 如果继续沿路径标记

假设当前已经标记路径到达了某个点  $x$ ，当前操作后的局面 SG 值为  $\text{cur}$ 。

如果继续从  $x$  走到它的某个未标记邻居  $y$ ，那么原来的  $\text{cur}$  中包含了子游戏：

$$F(y, x)$$

因为如果不继续走向  $y$ ，那么  $y$  这一整棵子树仍然是一个独立子游戏。

但现在我们选择继续标记  $y$ ，于是  $F(y, x)$  被替换成：

$$G(y, x)$$

因此新的局面值是：

$$cur' = cur \oplus F(y, x) \oplus G(y, x)$$

这就是本题最关键的转移。

## 求 $F(u, p)$

一个合法操作就是从  $u$  出发，沿着某条简单路径标记 1 到  $k$  个点。

每个终点都会对应一个操作后的 SG 值。设这些值组成集合  $S$ ，那么：

$$F(u, p) = \text{mex}(S)$$

其中  $\text{mex}(S)$  是不在集合  $S$  中的最小非负整数。

## 初始状态如何判断

如果初始点是  $s$ ，那么  $s$  已经被标记，剩下每个邻居  $v$  对应一个独立子游戏：

$$F(v, s)$$

所以初始局面的 SG 值是：

$$SG(s) = \bigoplus_{v \in \text{adj}[s]} F(v, s)$$

若：

$$SG(s) \neq 0$$

则 Alice 必胜；否则 Bob 必胜。

---

## 3. Trick 与陷阱

### Trick 点拨

这题的核心 Trick 是：

把“已标记连通区域外的每一棵树枝”看成一个独立 SG 子游戏。

然后用有向边  $(u, p)$  表示“从已标记点  $p$  向未标记子树  $u$  扩展”的状态。

另一个关键点是“替换式异或转移”：

$$cur' = cur \oplus F(y, x) \oplus G(y, x)$$

因为继续走向  $y$  时，本质上是把原先的子游戏  $F(y, x)$  替换成了标记  $y$  后的局面  $G(y, x)$ 。

## 易错点

1. 不能只看剩余点数对  $k+1$  取模。树上有分支，分支之间会产生 SG 异或。
2. 每次最多标记  $k$  个点，指的是点数，不是边数。所以 DFS 深度从 1 开始。
3. 叶子状态  $F(\text{leaf}, \text{parent})$  的唯一操作是标记这个叶子，操作后局面为 0，所以：

$$F(\text{leaf}, \text{parent}) = \text{mex}\{0\} = 1$$

1. 初始点  $s$  本身已经被标记，答案不是  $F(s, 0)$ ，而是：

$$\bigoplus_{v \in \text{adj}[s]} F(v, s)$$

1.  $\text{mex}$  的候选值可能来自异或，可能大于  $n$ 。但一个状态的合法操作数最多  $n$  个，所以 SG 值不会超过  $n$ ，实现时只需要记录  $0 \dots n$  范围内的候选值。

## 4. 代码实现 C++23

```
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct Solver {
    int n, k;
    vector<vector<int>> adj;
    vector<int> f, g;

    int id(int u, int p) {
        return u * n + p;
    }

    int F(int u, int p) {
        int &res = f[id(u, p)];
        if (res != -1) {
            return res;
        }

        int base = G(u, p);
        vector<char> vis(n + 1);

        auto dfs = [&](auto &&self, int v, int par, int dep, int cur) -> void {
            if (cur <= n) {
                vis[cur] = 1;
            }

            if (dep == k) {
                return;
            }
        };

        for (int v : adj[u]) {
            dfs(self, v, u, dep + 1, cur);
        }

        res = 0;
        while (vis[res]) res++;
        return res;
    }

    int G(int u, int p) {
        return f[id(u, p)];
    }
};
```

```

    }

    for (auto w : adj[v]) {
        if (w == par) {
            continue;
        }

        int nxt = cur ^ F(w, v) ^ G(w, v);
        self(self, w, v, dep + 1, nxt);
    }
};

dfs(dfs, u, p, 1, base);

res = 0;
while (res <= n && vis[res]) {
    res++;
}

return res;
}

int G(int u, int p) {
    int &res = g[id(u, p)];
    if (res != -1) {
        return res;
    }

    res = 0;

    for (auto v : adj[u]) {
        if (v != p) {
            res ^= F(v, u);
        }
    }

    return res;
}

string work() {
    f.assign(n * n, -1);
    g.assign(n * n, -1);

    string ans;

    for (int s = 0; s < n; s++) {
        int x = 0;

        for (auto v : adj[s]) {
            x ^= F(v, s);
        }

        ans += char('0' + (x != 0));
    }
}

```

```

    }

    return ans;
}
};

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    int T;
    cin >> T;

    while (T--) {
        Solver sol;
        cin >> sol.n >> sol.k;

        sol.adj.assign(sol.n, {});

        for (int i = 0; i < sol.n - 1; i++) {
            int u, v;
            cin >> u >> v;
            --u;
            --v;

            sol.adj[u].push_back(v);
            sol.adj[v].push_back(u);
        }

        cout << sol.work() << '\n';
    }

    return 0;
}

```

## 复杂度分析

每个有向边状态  $(u, p)$  只会计算一次。

计算  $F(u, p)$  时，会枚举从  $u$  出发、不经过  $p$ ，长度不超过  $k$  的路径终点。最多不会超过该有向边对应连通块的大小。

所有有向边对应连通块大小之和为：

$$O(n^2)$$

因此总复杂度为：

$$O(n^2)$$

空间复杂度为：

$$O(n^2)$$

符合：

$$\sum n \leq 2000$$

的限制。

---

## 5. 总结与启示

### 难度评分

预估 Codeforces Rating：

2300

如果熟悉 SG 函数，这题主要难在状态抽象；如果不熟悉 SG 博弈，这题会非常难切入。

### 核心启示

树上扩展型博弈的关键是：把已占据的连通区域删掉后，剩余挂枝会变成独立子游戏，胜负由 SG 异或决定。

### 同类习题推荐

1. AtCoder AGC017 D - Game on Tree  
经典树上 SG 博弈。
2. Codeforces 812E - Sagheer and Apple Tree  
树结构与异或博弈结合。
3. AtCoder DP Contest K - Stones  
用来巩固 mex、SG 值和取石子类博弈的基础。